SmartPID M5 PRO in *Proof*

MQTT Power-Control Investigation & Feature Recommendations

Prepared for Davide / SmartPID • May 2026

1. Introduction and Goal

Proof is a self-hosted, hobbyist distillery application that can drive a SmartPID M5 PRO over MQTT. The M5 PRO controls the heating elements on a hobbyist still: a primary element on an SSR for fine power control, and an auxiliary element on a contactor used only to accelerate heat-up. For logic limitations that will become apparent later in this report, *Proof* owns the run logic and issues commands to the M5; the M5 executes them and runs autonomously if the software or the network fails.

The goal of this investigation was to answer two questions. First, can the M5 PRO be commanded, over MQTT, to operate a heating element at specific and changing operator-chosen power levels during a run, independent of any temperature setpoint? And second, what level of integration into *Proof* is currently possible and how could it be further integrated in the future?

Distillation is fundamentally a **power-controlled** process, not a temperature-controlled one. During a run the operator sets a power level to achieve a desired take-off rate and nudges it as the run progresses; the boiler temperature is an **output** of that choice, not the target. This is the opposite of the M5 PRO Standard and Advanced modes, which both control **to a temperature**. This investigation was to utilize the M5 PRO — which is otherwise an excellent fit for mashing and brewing control — to serve the power-first use case of distillation control.

The short answer: yes, it can, but when integrated with other systems such as *Proof*, a workaround is required which I describe below. It works reliably but the workaround is required because the device has no native MQTT-communicable power-control functions, and this report documents both the investigation and a recommendation for how a small addition would make the M5 PRO a first-class fit for an entire class of distillation applications it currently serves only indirectly.

2. Methodology

All testing was performed against a physical M5 PRO unit on a self-hosted Eclipse Mosquitto 2.1.2 broker on a local network — no cloud involvement at any point. Commands were published with standard MQTT tooling to the device's command topic, and telemetry was observed on the dynamic and events topics.

Physical output was verified independently of telemetry. A voltmeter on a DC output terminal confirmed actual PWM duty cycle (the 3500 ms PWM period makes duty cycles directly observable), and relay state was confirmed. This let me distinguish what the device reports from what it physically does — a distinction that turned out to matter.

Test conditions: temperature probes connected to both channels reading ambient; no heating element or live load attached (outputs observed as signals only); device powered from AC mains. The device unit was configured in Fahrenheit, so all setpoint values below are in degrees Fahrenheit.

3. Investigation

3.1 The core challenge: no direct power command

My first hypothesis was that a direct power command would exist. It does not. I tested `{"CH1 pwm": N}` in both monitor and standard modes; it is silently ignored in both. The `pwm` field is read-only telemetry — it reports the PID's computed demand, but cannot be written. There is no documented command that I could find to set an output to a chosen percentage directly.

3.2 Monitor mode cannot be driven over MQTT

Monitor mode is exactly the right concept for manual power control — it lets you select an output, set a PWM level, and activate it. On the device, by hand, it does precisely what would be needed for power control for distillation. But over MQTT it is inert: sending `{"CH1 maxpwm": N}` in monitor mode produces no output activation, and telemetry stays minimal (`time, temp, unit, runmode` only). Monitor-mode output control appears to require physical button interaction and is not exposed to the MQTT command path.

3.3 Enabling MQTT power control: pinned setpoint + maxpwm

The breakthrough was realizing that `maxpwm` — an MQTT-communicable PWM ceiling — physically clamps the output in PID mode. With the setpoint pinned far above the process temperature, the PID will demand 100% PWM, so the physical output equals the `maxpwm` ceiling. In effect, **maxpwm becomes a direct power command**, as long as the setpoint is held unreachably high.

The confirmed sequence:

```
{"stop": true}
{"start": "standard"}
{"CH1 SP": 842}      // pin setpoint at device max so PID always demands 100%
{"CH1 maxpwm": 40}   // physical output = 40% duty cycle
```

Changing power mid-run requires only `{"CH1 maxpwm": N}` — no stop, no restart. The change takes effect within the current PWM cycle.

A critical detail for anyone reading telemetry: under a pinned setpoint, the `pwm` field always reads 100 (it is the pre-ceiling PID demand). The `maxpwm` field is the value that reflects actual output. Any software displaying “power” must read `maxpwm`, not `pwm`.

3.4 Verifying it physically

With the setpoint pinned at 842°F, I confirmed on a voltmeter that the output duty cycle tracked the commanded maxpwm:

Command	Expected duty (3500 ms period)	Voltmeter observation
{"maxpwm": 30}	ON ~1.05 s / OFF ~2.45 s	Cycling confirmed at ~30%
{"maxpwm": 60}	ON ~2.1 s / OFF ~1.4 s	Noticeably longer ON, ~60%
{"maxpwm": 0}	Constant off	Constant 0 V

This is a clean, reliable, proportional power-control mechanism. It works. The rest of the investigation was about whether I could also get the device to enforce a safety-relevant cutoff autonomously.

3.5 The auxiliary-element cutoff: where the workaround runs out

The envisioned setup is a still with two heating elements: a primary for distillation, and an auxiliary element used only during heat-up, which then drops once the boiler nears boiling, leaving the primary element in fine control. I wanted the M5 PRO to enforce that cutoff itself — set the auxiliary channel's setpoint to the cutoff temperature and let the device open the relay when the temperature is reached — so the cutoff would survive even if *Proof* were offline.

This does not work cleanly, for two reasons that compound:

1. **PID modulates in the approach zone.** With the auxiliary channel's setpoint at the cutoff temperature, the PID reduces output proportionally across a band below the setpoint. On a relay driving a contactor, that means the contactor time-proportions — clicking on and off many times — as the temperature crosses the band.
2. **ON/OFF mode would fix the clicking but breaks the power mechanism.** Switching the global algorithm to ON/OFF gives clean binary relay behavior — but it also disables the maxpwm time-proportioning the primary element depends on. In ON/OFF mode, maxpwm acts only as a binary gate (>0 allows ON, 0 forces OFF); it no longer seemed to produce a duty cycle. And because the PID-versus-ON/OFF algorithm is **global, not per-channel**, I cannot run the primary in PID and the auxiliary in ON/OFF simultaneously.

During my to-date testing of the device, it can either give me proportional power control on the primary (PID, but with contactor chatter on the auxiliary cutoff) or clean relay switching on the auxiliary (ON/OFF, but no proportional power on the primary) — never both.

3.6 Relay Control

As a potential bonus, I attempted to control the relays over MQTT to enable software-driven alarms and reminders — for example, a panel-mounted buzzer for cut alarms during distillation or mash timers during mashing. No relay command could be found or guessed during the investigation; relays can only be actuated indirectly, by writing a temperature setpoint above or below the current reading to flip the channel between heating and cooling states. Any integrated relay work therefore has to repurpose a control loop to do something it was not really meant to do.

This indirect-only relay behavior also limits the staged safety design (see §5). Ideally a condenser-monitoring unit would fire two relays off a single probe — a warning buzzer at one threshold and a panel shutdown at a higher one. But each relay is tied to its own channel and setpoint, so two thresholds on one probe is not cleanly possible without dedicating both channels (and thus the probe) to the cutoff function. In practice I expect to make the final shutdown a hardware relay action on a second unit, and drive the warning buzzer as a software alarm from *Proof* instead. Direct, independent relay commands (see §6.2) would remove the need for the workaround entirely and make staged hardware alarms straightforward.

4. Results

The complete set of confirmed findings:

Capability	Result
Direct PWM command (set output to N%)	Not available — pwm is read-only; the command seemed to be silently ignored
Power control via pinned SP + maxpwm (PID mode)	Confirmed — maxpwm = physical output %, verified on voltmeter
Mid-run power change via maxpwm alone	Confirmed — immediate, no restart
Mid-run setpoint write	Confirmed — reflected in next telemetry tick
Required for power mechanism	Global PID + Heating mode (ON/OFF disables duty cycling)
Reading actual power from telemetry	Read maxpwm, not pwm (pwm = 100 under pinned SP)
Direct relay command	Not available — only via setpoint above/below temperature
Relay state in telemetry or trigger over MQTT	No dedicated field — inferred from mode (heating/cooling)
Monitor-mode output activation over MQTT	Not available — required physical buttons
Device-enforced relay cutoff at a temperature	PID chatter vs. ON/OFF trade-off, global mode
Autonomy through broker/network loss	Confirmed — holds last command indefinitely, resumes cleanly
Auto Resume after power cycle	Confirmed, but setpoint reverted to stored default (see note)

One quirk worth flagging independently of any feature request: after a power cycle with Auto Resume enabled, the process auto-resumed but the setpoint reverted to the device's stored default (131°F on my unit), **not** the last value commanded over MQTT. Software would have to detect the `"power restored"` event and re-send the setpoint and maxpwm. This may be intended, but it surprised me, and I was not sure if this was a one-off error or if this was the intended functionality.

5. Runup Cutoff Control Methods

For power control, I adopted the pinned-setpoint + maxpwm mechanism. It is reliable, proportional, and survives network loss. *Proof* pins the primary channel's setpoint at 842°F, commands power as maxpwm, reads maxpwm back for display, and re-sends state on the power-restored event. This is a solid method of MQTT control.

For the auxiliary-element cutoff, since the device cannot enforce it cleanly on the control unit itself, I split the function into two layers for safety:

- **Operational cutoff (software-commanded):** *Proof* sends `{"CH2 maxpwm": 0}` when the boiler reaches the run-up temperature. This produces a single clean relay open, and its failure mode is benign — if *Proof* is offline for a brief moment, and as long as there is an additional safety cutoff (see below), the auxiliary element simply runs long.
- **Safety cutoff (hardware, device-autonomous):** a second M5 PRO unit serves as an independent hardware safety layer. While the first unit drives the elements, the second unit monitors the condenser output temperature and acts entirely on its own — independent of *Proof*, the first M5 PRO, the MQTT broker, and the network. It a final do-not-exceed level (e.g. 170 °F) relay to open a circuit in series with the **entire panel**, dropping **both** elements at once. This is a sound method but it is worth being honest that I arrived at it partly by working around the device rather than with it.

6. Recommendation: A Native "Power Management" Mode

Everything above is a workaround for one missing mode. The M5 PRO has two run modes today, and **both control to a temperature**: Standard (fixed setpoint) and Advanced (ramp/soak profile). Distillation actually needs a third mode that allows for MQTT control of **power** directly. I would suggest it sit right alongside the others in the Start menu:

Mode	Controlled variable	Use case
Standard	Temperature (fixed setpoint)	Thermostatic hold
Advanced	Temperature (ramp/soak profile)	Mashing, multi-stage processes
Power Management (proposed)	Output power (%)	Distilling, direct-power applications

The device already does everything required for this mode internally — the maxpwm clamp, the time-proportioning, the autonomy on disconnect. A Power Management mode would simply expose that capability directly, instead of requiring users to pin a fake setpoint and repurpose the PWM ceiling as a power command.

6.1 Proposed functions

- **Direct power setpoint per channel.** The user sets a power percentage (0–100%), not a temperature. The output time-proportions at that duty cycle using the existing PWM period — exactly what the maxpwm clamp already produces, but as the primary, intended control value rather than a ceiling.
- **No pinned-temperature trick.** The setpoint field would not be needed for the power loop at all, removing the 842°F workaround and the confusing pwm-reads-100 telemetry behavior.
- **Temperature limit.** A user could optionally set a maximum temperature at which the device backs off or cuts power — a genuine safety/quality ceiling layered on top of power control. For distilling this would let the controller defend a vapor-temperature limit while otherwise holding power.
- **Per-channel mode would be ideal.** The single biggest limitation I hit was that the PID-versus-ON/OFF algorithm is global. If a Power Management channel could coexist with an ON/OFF channel — or if Power Management simply offered a clean per-channel "power with optional on/off cutoff" — the auxiliary-element cutoff problem in §3.5 would disappear entirely.

- **Telemetry would report commanded power directly.** A field that reports the commanded power level (today only maxpwm carries this, and only under the pinned-SP trick) would make integration unambiguous.

6.2 Recommended API additions

Independent of the full mode, three command-level additions would dramatically simplify MQTT integration. Each maps to a specific gap confirmed in testing:

Proposed command	What it would do	Gap it closes
{"CHx power": N}	Set output to N% duty cycle directly, without pinning a setpoint	pwm is read-only today; maxpwm-with-pinned-SP is the only path
{"CHx relay": true/false}	Drive a relay output ON/OFF directly	Relays can only be toggled via setpoint above/below temperature
{"CHx output": "on/off", "level": N}	Activate a monitor-style output over MQTT	Monitor-mode output control requires physical buttons today

Of these, `{"CHx power": N}` is by far the most valuable — it is the single command that would make the entire workaround in §3.3 unnecessary. A direct relay command would be the second; it would let users drive an alarm indicator or an auxiliary actuator or solenoid without manipulating a control loop to do it.

6.3 Future development

The M5 PRO is genuinely excellent for this application in every respect: it is autonomous on disconnect, it has dual channels, it speaks plain MQTT to a self-hosted broker with no cloud dependency, and its hardware already produces clean proportional output. Distilling is a large and enthusiastic hobbyist market, and every still controller faces the exact power-control problem described here: a lack of a configurable, network managed and software commanded power management mode. A Power Management mode would let the M5 PRO serve that market directly — and, as far as I can tell, it asks almost nothing new of the hardware, only of the firmware's control logic and command parser.

I am happy to test any firmware build of such a mode against my setup and report back in the same detail. Thank you for building a controller flexible enough that this workaround was even possible — that flexibility is the reason the M5 PRO is the right device for this project!